FumiBot V2: An Innovative Cleaning Solution for Flat Surface Mopping

An Investigatory Project presented to:

The faculty of the Junior High School Department
of Saint Joseph School (Naga City), Inc.

For the Culminating Task of the following subjects:
SCIENCE 10
INFORMATION and COMMUNICATION TECHNOLOGIES 10
(ICT)

Proposed by:

Project Manager
Penilla, Janaya Therese G.
Uy, Rover Rhys T.

Project Designer
Co, Ethan Gabriel P.
Dimayuga, Rain Hayden G.
Tomosada, Josiah M.

Document Manager and Researcher
Carcido, Ryza Chloe K.
Medroso, Misha Jeandra J.
Pereña, Noah P.
Peyra, Roma Sophiya B.

Date of Submission
February 13, 2025

Abstract

In this present time, people are having trouble finding the correct cleaning materials to clean and sanitize spaces that are difficult to reach or out of sight In this period of time where outbreaks of diseases are frequent, caused by viruses and bacteria, people have become more careful of their surroundings to ensure their safety and health. Here comes the FumiBot. FumiBot is easy to use and allows users to manually maneuver the robot from far distances using a phone.This robot can also move automatically just by pressing the object avoidance mode. The robot uses a sensor to check its distance from ventilation shaft walls and turns in a different direction when nearby to avoid damaging itself. Because of these features, users can avoid being in the same room during disinfection. And lastly, it has an on-off button, allowing users to activate or deactivate the robot. Upon purchase, it comes with clear instructions and labels to guide users on how to use FumiBot. This bot is essential in avoiding the spread of infectious diseases. It's simple-to-use which may cause them to bring home unwanted germs at the end of the day. With the use of FumiBot, the bacteria and debris causing illnesses inside workplaces, bedrooms, and other personal spaces that people frequently stay in can be minimized.

Table Of Contents

CHAPTER 1

I. Introduction

Introducing FumiBotV2, the innovative disinfectant robot designed to guarantee hygiene and safety in response to developing diseases. With its user-friendly features, FumiBotV2 provides both individual and community disinfection that is both practical and efficient. The concept behind FumiBotV2 is to build a disinfection robot that can remove dirt and prevent germs and viruses from growing. People spend the majority of their time outside engaging in various activities with numerous people, which may cause them to bring home unwanted germs and bacterias at the end of the day. With the help of FumiBotV2, the bacteria and viruses causing illnesses inside workplaces, bedrooms, and other personal spaces that people frequently stay in can be minimized.

II. Statement of the Problem

This study aims to develop a bot designed to mop any flat surfaces. It addresses the need for a disinfectant robot that cleans surroundings in a semi-autonomous manner, lowering the risk of infection. In order to mitigate the spread of diseases and increase public health. The following queries are addressed in this study:

1. Is the semi-autonomous cleaning robot capable of sanitizing flat areas to reduce the spread of diseases?
2. Does the semi-autonomous cleaning robot save more time compared to traditional cleaning methods?
3. Is the semi-autonomous cleaning robot a long-term solution for controlling the spread of disease

III. Engineering Goals

Scientifically, the FumiBot V2 integrates advanced components such as ultrasonic sensors, a motor control system, and disinfectant spray components to ensure comprehensive cleaning while minimizing human interaction. Socially, the device mainly

addresses the increasing demand for automated sanitation solutions, particularly in public spaces, to enhance public health and safety by providing an effective, easy-to-use tool for continuous surface disinfection

IV. Significance of the Study

With the increase of disease spreading in the world, the FumiBot V2 limits physical interaction between people while cleaning and disinfecting. With our bot, less transmission of diseases and infection will occur, as our bot can be controlled using a controller installed in your phone. Our bot also mainly targets the demand for automated sanitation solutions for public places, such as hospitals, to enhance safety. Our bot helps prevent the widespread spread of dirt and bacterias that may cause diseases.

Because it is managed by using via phone, users may no longer have any physical contact in cleaning and instead control it. Reducing the need for physical work Furthermore, Fumibot V2 responds to cleaning in high risk settings such as at the hospital, restaurants, and more public places. As a result, the FumibotV2 not only enhances public health and safety of the users but also provides a sustainable way to maintain cleanliness in busy places, a long-term solution for keeping busy areas clean.

V. Scope and Limitations

The FumiBot V2 main use is to clean surroundings which results in less spreading of diseases. You can let the bot clean areas by itself or you can use the bot to clean the area manually.

However, our bot can be only used on flat surfaces. These flat surfaces include the floor and pathways of a house, hospital or room. Rough surfaces, such as rocky roads, mats and rugs will result in the bot not as intended and with most effectiveness. It will slow down its process and make the cleaning time more slower.

Additionally, you must also refill the disinfectant liquid from time to time because no liquid might not come out if the container is empty. The user should also check the battery, as the battery only lasts for at least 18 hours.

Furthermore, if you want to control the bot manually, you need to connect to the ESP32S then go to our bot's website to fully control its movement. It also has a distance limit in order to control the bot. The distance to control the bot is within 20 meters.

CHAPTER 2

VI. Review of Related Literature and Systems

One of the existing cleaning devices related to our robot is a Mop. Originally, to use a mop you have to dip the mop into a disinfectant liquid, dry it, then wipe it to a surface. However, our bot does not need to do extra things. As our bot wipes the surface then sprays the disinfectant liquid on it.

The closest cleaning device to our bot is a Roomba. A Roomba almost has the same function as our bot, it navigates through obstacles using sensors and cleans and disinfects the surface. However, our bot can be manually controlled using a controller installed in your phone. Unlike the Roomba, which is only autonomous, you can control our bot's direction, as long as it is a flat surface.

CHAPTER 3

VII. Materials

Component	Function
Arduino Uno R3 Compatible Board	This is the main controlling board.
L293D Motor Control Shield	This is the main motor controller of the movement of the motors.
ESP32S with Cam & External Antenna	This is the main communication module and camera.
12V Battery	This will power all the electrical components in the robot.
Ultrasonic Sensor	This sensor will measure the distance from itself to an object in front of it.
Servo Motor (180°)	This motor will turn the ultrasonic sensor from left to right (180°).
Hobby Gearmotor	This motor will control the movement of the wheels.
Disinfectant Container/Storage	This is the storage for the disinfectant liquid.
Water Pump	This will pump the disinfectant liquid out of the disinfectant container/storage.
Hose	This allows the disinfectant liquid to flow to the nozzle.

Nozzle	This will spray the disinfectant liquid almost into mist form.
Rectangular Mop Head	This will wipe the sprayed disinfectant liquid.
Jumper Wire	This will be used to connect components either to each other or to the Arduino Uno R3 Compatible Board.
Resistor	This will regulate the electrical current flow in the circuit.
Filament	This will be used in 3D printing parts for the robot.
Cable Tie	This will be used to group wires together.
Soldering Iron	This, with lead, will solder wires and pins.
Lead	This will connect wires and or pins.

VIII. Design/Construction Process

In order to successfully create a functional robot, our team underwent many procedures. First of all, planning, we shared ideas, thoughts and concepts with each other. In the end, we agreed on improving a previously made bot. After the agreement, Assigning of Tasks, in order to make a fully functional improved bot, we split our group into teams. The designers are in charge of making a design of the bot in order for the bot to work properly. Then, the researchers, they are in charge of researching functions that will be added in the improved bot.

Our designers based the design on the requirements that are needed to be achieved. The bot should be able to move without weight be a problem, a body that is sturdy, and a design that looks appeasing.

Firstly, we separated the water container from the other components, such as the Hobby Gearmotors, Ultrasonic Sensor and the ESP32S, in order to avoid any malfunctions due to broken wirings and components. Then, we also placed the water pump in the back and the container in the front in order for the bot to have balance while it is working. Not only it helps in balance it also helps in the appearance of the bot. We also added an ESP32S Camera to track the movements of the bot while you are in a far distance .

The parts were all 3D printed using PLA filament. Each part of the design was tailored to be compatible with one another. The bot uses several adhesives to connect each part together, such as Silicone adhesive, Epoxy, and 4mm Screws. We used Silicone to ensure no liquid would leak from the water container, Epoxy to mend parts such as the Front section of the Bot to the Body, and 4mm screws to attach the top of the bot to the main compartment to provide protection yet still maintaining easy access to the components.

Each part of the body was carefully designed to house the components without any complications. We first inserted the Water to pump, this was to check if any liquid would be an issue for the electrical components. Next, the Ultrasonic Sensor, ESP32(CAM), and Servo motor was installed, this was to see if the bot has visibility to see any obstacles that would hinder the bot's performance.The L293D Motor shield and DC motors were integrated last, this was to see if the Motor functions of the bot needed to be improved.

CHAPTER 4

IX. Results and Discussion

The team conducted a series of tests to assess its efficiency, capabilities and shortcomings. The main target of these tests was to determine if the FumiBotV2 could disinfect and clean surfaces by wiping away dirt, spraying disinfectant, and ensuring proper sanitation. Each part of FumibotsV2s body was carefully examined to look for any problems in the inner and outer part of the body. The Hobby Gearmotors performed good as expected, allowing smooth movement. The disinfecting mechanism, including the nozzle and mop head, completed its designated tasks. The nozzle sprayed the disinfectant onto the surface, and the mop head wiped away contaminants, Letting us know that the functions of the FumiBotv2 works.

To further evaluate the FumiBot V2, the device was tested on a flat surface scattered with dirt. The bot was placed on the surface and activated using a mobile controller. As the bot moved across the surface, it was observed that the cleaning mechanism removed the dirt before spraying the disinfectant. The mobility of the bot was smooth, and the cleaning process was consistent, ensuring that the surface was both cleaned and sanitized. This experiment demonstrated that the FumiBot V2 could function autonomously while maintaining a high level of cleaning.

CHAPTER 5

X. Conclusion

FumiBot V2 is a **semi-autonomous cleaning robot** made to **disinfect and clean flat surfaces efficiently** while **lessening human contact with bacteria and debris**. It has two operating modes: manual **control from a mobile app** and **autonomous mode; it uses** the **ultrasonic sensors to avoid obstacles**. The Tests show that the robot can **sanitize surfaces**, achieving its goal of **lessening infection risks** while making cleaning **easier and safer**.

Its **use** across **hospitals, offices, schools, and homes**, where sanitation is key. In **healthcare** it can help **lessen hospital-acquired infections**. **Businesses** gain from **reduced need for manual cleaning labor**, whilst **schools** can have a safe learning environment. Homeowners can also have a simple yet effective cleaning method

Nevertheless,The Fumibot does have **limitations**. The robot **only works on flat surfaces**, limiting the use on **carpets and even uneven floors**. The **disinfectant tank needs to be refilled by hand**, the **manual mode is only limited to a 20-meter range**. In addition to battery **life limits it may** need to be charged on a regular basis. All these **weaknesses can be improved** by increasing **battery efficiency, increasing surface distance, and increasing disinfectant capacity**.

Overall, the **FumiBot V2 successfully achieves its purpose** as a **reliable, automated cleaning solution**. With further improvements, it has the potential to **set new standards in robotic sanitation**, making cleaning **faster, safer, and more effective**.

XI. Recommendations

For homeowners & renters

Homeowners and renters can use FumiBotV2 to maintain clean and hygienic living spaces with minimal effort. Frequent mopping reduces allergens in the house and dirt accumulation by removing dust, filth, and spills. FumiBotV2 is guaranteed to be a beneficial choice because of dirt absorption. Maintaining a mopping schedule prolongs the life of floors and keeps them looking new, such as once a week for low-traffic areas and more regularly for kitchen and entryways.

For restaurant and healthcare workers

Restaurant workers need to mop floors frequently to maintain cleanliness and prevent slips, especially in high-traffic areas like the kitchen and dining space. Grease, food spills, and tracked in debris all create risks, making regular cleaning vital for both hygiene and safety. Maintaining a hygienic workplace and removing tough residues are made easier by using FumiBotV2 which is a flat surface cleaning mop. To prevent disturbing customers, staff should mop during slack hours and perform a thorough cleaning at closing. Maintaining a spotless floor also helps adhere to health and safety standards, avoiding infractions and guaranteeing a positive reputation.

In order to keep a clean and sterile atmosphere and lower risk of infection, healthcare personnel must mop floors on a frequent basis. Specialized disinfection solutions are needed in hospitals and clinics to get rid of bacteria, viruses, and other impurities. When used in conjunction with a color-coded cleaning system, FumiBotV2 guarantees effective cleaning while reducing cross-contamination. A systematic pattern should be followed when mopping, with high-risk locations such as patient rooms and hallways being cleaned many times a day. In healthcare settings, maintaining floors properly promotes overall cleanliness, regulatory compliance, and patient safety.

XII. References

1. https://doi.org/10.1186/s13756-020-00878-4*cs*. (n.d.). IFR International Federation of
   Robotics. https://ifr.org/case-studies/cleaning-robots-reduce-infections

2. Al-Farsi, A., & Hassan, T. (2023, November 20). *Evaluation of Autonomous Cleaning*
   Robots in Reducing Hospital Acquired Infections: A Comparative Study. https://vectoral.org/index.php/IJSICS/article/view/77/

3. Robotic *Technology: Cleaning Trends for Healthcare in 2023*. (n.d.). ICE Cobotics.
   https://icecobotics.com/blog/robotic-technology-clhttps://vectoral.org/index.php/IJSICS/article/view/77/

4. Schahawi, M. D. E., Zingg, W., Vos, M., Humphreys, H., Lopez-Cerero, L., Fueszl, A.,

Zahar, J. R., & Presterl, E. (2021). Ultraviolet disinfection robots to improve hospital cleaning: Real promise or just a gimmick? *Antimicrobial Resistance and Infection Control*, *10*(1). https://doi.org/10.1186/s13756-020-00878-4

XIII. Appendices

5.4.1: Appendix A

Source code for the Robot:

```
/****************************************************************************
***

Include external libraries

*****************************************************************************
**/

#include "Objects.h"
#include <SoftwareSerial.h>

/****************************************************************************
***

Define pin assignments

*****************************************************************************
**/

// ESP32 Pins
#define pRX            A0
#define pTX            A1

// Water Pump Pin
#define pPump          A2
```

```
// Ultrasonic Sensor Pins
#define pTrig              A3
#define pEcho              A4

// Servo Motor Pin
#define pServo             10

// Hobby Gearmotor Pins
#define pMotor1            3
#define pMotor2            6
#define pMotor3            5
#define pMotor4            11

// L293D Motor Driver Shield Pins
#define DIR_CLK            4     // Data input clock line
#define DIR_EN             7     // Equip the L293D enabling pins
#define DATA               8     // USB cable
#define DIR_LATCH          12    // Output memory latch clock

/*********************************************************************
***

Constants definition for direction commands

*********************************************************************
**/

#define cmd_AutoOn         1
#define cmd_AutoOff        2
#define cmd_PumpOn         4
#define cmd_PumpOff        5
#define cmd_Forward        212
#define cmd_Backward       43
#define cmd_Left           135
#define cmd_Right          120
#define cmd_rotateLeft     153
#define cmd_rotateRight    102
#define cmd_Stop           0
```

```
#define cmd_UpperLeft          132
#define cmd_UpperRight          80
#define cmd_LowerLeft           3
#define cmd_LowerRight          40

#define Left                  false
#define Right                  true

/*******************************************************************
***

Define system variables

*******************************************************************
**/

bool nextDir = Left;
byte turnCounter;

int avoidDistance = 10;         // Minimum distance from obstacle
int leftDistance;
int rightDistance;

/*******************************************************************
***

Create global class instance

*******************************************************************
**/

Pump myPump;
Motor myMotor;
Scanner myScanner;
SoftwareSerial ESP32(A0, A1);

/*******************************************************************
***
```

Function: UTurn(turnSide)
Parameters: turnSide: whether to turn left or right

Make a U-turn to given direction

****
**/

```
void UTurn(bool turnSide){
 // Alternate U-turn direction
 nextDir = !turnSide;
 int rotateDir;

 if(turnSide == Left){
  rotateDir = cmd_rotateLeft;
 } else{
  rotateDir = cmd_rotateRight;
 }

 myMotor.move(rotateDir);
 delay(400);
 myMotor.move(cmd_Stop);
 delay(250);
 myMotor.move(cmd_Forward);
 delay(200);
 myMotor.move(cmd_Stop);
 delay(250);
 myMotor.move(rotateDir);
 delay(400);
 myMotor.move(cmd_Stop);
 delay(250);
}
```

/***
***

Function: AutoMode

Run algorithm for autonomous movement

```
*************************************************************************
**/

void AutoMode(){
  if(myScanner.frontDistance <= avoidDistance){
    myMotor.setSpeed(255);
    myMotor.move(cmd_Stop);
    delay(250);
    myScanner.scan(15);
    int leftDistance = myScanner.frontDistance;
    myScanner.scan(165);
    int rightDistance = myScanner.frontDistance;
    myScanner.scan(90);
    if(leftDistance <= avoidDistance && rightDistance > avoidDistance){
      UTurn(Right);
      delay(200);
      turnCounter = 0;
    } else if(leftDistance > avoidDistance && rightDistance <= avoidDistance){
      UTurn(Left);
      delay(200);
      turnCounter = 0;
    } else if(leftDistance > avoidDistance && rightDistance > avoidDistance){
      if(leftDistance < rightDistance){
        switch(turnCounter){
          case 0:
            UTurn(Left);
            turnCounter = 1;
            break;

          case 1:
            UTurn(Right);
            turnCounter = 2;
            break;

          case 2:
            UTurn(Left);
```

```
        turnCounter = 0;
        break;
      }

      delay(200);
    } else if(leftDistance > rightDistance){
      switch(turnCounter){
        case 0:
          UTurn(Right);
          turnCounter = 1;
          break;

        case 1:
          UTurn(Left);
          turnCounter = 2;
          break;

        case 2:
          UTurn(Right);
          turnCounter = 0;
          break;
      }
      delay(200);
    } else{
      UTurn(nextDir);
      delay(200);
    }
  }
 } else{
  myScanner.scan(90);
  myMotor.setSpeed(100);
  myMotor.move(cmd_Forward);
  delay(200);
 }
}

// Initialize system
void setup(){
 ESP32.begin(115200);
 // Serial.begin(115200); -> debugging
```

```
  myPump.attach(pPump);



  myMotor.attach(pMotor1, pMotor2, pMotor3, pMotor4, DIR_CLK, DIR_EN, DATA,
DIR_LATCH);
  myScanner.attach(pTrig, pEcho, pServo);

  myPump.stop();
  myMotor.move(cmd_Stop);
}

// Main program
void loop(){
  byte cmd;

  // Wait for command from ESP32 Cam
  if(ESP32.available()){
    cmd = ESP32.read();
    //Serial.print(cmd); -> debugging
  }

  myMotor.setSpeed(180);

  //  Select action
  switch(cmd){
    // Switch pump ON
    case cmd_PumpOn:
      myPump.start();
      break;

    // Switch pump OFF
    case cmd_PumpOff:
      myPump.stop();
      break;

    // Switch auto movement ON
    case cmd_AutoOn:
      AutoMode();
```

```
      break;

    // Switch auto movement OFF
    case cmd_AutoOff:

myMotor.move(cmd_Stop);
      break;

    // Move forward
    case cmd_Forward:
      myScanner.scan(90);
      if(myScanner.frontDistance > avoidDistance){
        myMotor.move(cmd_Forward);
      } else{
        myMotor.move(cmd_Stop);
      }
      break;

    // Move backward
    case cmd_Backward:
      myMotor.move(cmd_Backward);
      break;

    // Move left
    case cmd_Left:
      myScanner.scan(15);
      if(myScanner.frontDistance > avoidDistance){
        myMotor.move(cmd_Left);
      } else{
        myMotor.move(cmd_Stop);
      }
      break;

    // Move right
    case cmd_Right:
      myScanner.scan(165);
      if(myScanner.frontDistance > avoidDistance){
        myMotor.move(cmd_Right);
      } else{
        myMotor.move(cmd_Stop);
```

```
      }
      break;



// Rotate left
    case cmd_rotateLeft:
      myMotor.move(cmd_rotateLeft);
      break;

    // Rotate right
    case cmd_rotateRight:
      myMotor.move(cmd_rotateRight);
      break;

    // Move upper left
    case cmd_UpperLeft:
      myScanner.scan(45);
      if(myScanner.frontDistance > avoidDistance){
        myMotor.move(cmd_UpperLeft);
      } else{
        myMotor.move(cmd_Stop);
      }
      break;

    // Move upper right
    case cmd_UpperRight:
      myScanner.scan(135);
      if(myScanner.frontDistance > avoidDistance){
        myMotor.move(cmd_UpperRight);
      } else{
        myMotor.move(cmd_Stop);
      }
      break;

    // Move lower left
    case cmd_LowerLeft:
      myMotor.move(cmd_LowerLeft);
      break;
```

```
    // Move lower right
    case cmd_LowerRight:
      myMotor.move(cmd_LowerRight);
      Break;


    // No pressed button
    default:
      myMotor.move(cmd_Stop);
      break;
  }
}
```

Source Code for the Classes used:

```
#include <Servo.h>

/************************************************************************
***

Class: Pump
The water pump system

*************************************************************************
**/

class Pump {
  private:
    byte pin;

  public:
    bool isON;

    Pump(){}      // Class constructor

    // Assign pin for the pump
    void attach(byte _pin){
      pin = _pin;
      pinMode(pin, OUTPUT);
    }
```

```
    // Switch pump ON
    void start(){

     digitalWrite(pin, HIGH);

      isON = true;
    }

    // Switch pump OFF

void stop(){
      digitalWrite(pin, LOW);
      isON = false;
    }
};

/*********************************************************************
***

Class: Motor
A set of 4 hobby gearmotor system

*********************************************************************
**/

class Motor {
  private:
    byte pin1, pin2, pin3, pin4;
    byte pinCLK, pinEN, pinDATA, pinLATCH;
    byte speed = 255;

  public:
    Motor(){}     // Class constructor

    // Assign pins for the motors
    void attach(byte _pin1, byte _pin2, byte _pin3, byte _pin4, byte _pinCLK, byte
_pinEN, byte _pinDATA, byte _pinLATCH){
      pin1 = _pin1;
      pin2 = _pin2;
```

```
    pin3 = _pin3;
    pin4 = _pin4;

    pinCLK = _pinCLK;
    pinEN = _pinEN;

    pinDATA = _pinDATA;
    pinLATCH = _pinLATCH;

    pinMode(pin1, OUTPUT);
    pinMode(pin2, OUTPUT);
    pinMode(pin3, OUTPUT);
    pinMode(pin4, OUTPUT);

    pinMode(pinCLK, OUTPUT);
    pinMode(pinEN, OUTPUT);
    pinMode(pinDATA, OUTPUT);

pinMode(pinLATCH, OUTPUT);
  }

  // Set rotation speed of motors
  void setSpeed(byte newSpeed){
    speed = newSpeed;
  }

  // Move the motor to a defined direction (dir)
  void move(byte dir){
    byte _speed = (dir == 0) ? 0 : speed;

    analogWrite(pin1, _speed);
    analogWrite(pin2, _speed);
    analogWrite(pin3, _speed);
    analogWrite(pin4, _speed);

    digitalWrite(pinLATCH, LOW);
    shiftOut(pinDATA, pinCLK, MSBFIRST, dir);
    digitalWrite(pinLATCH, HIGH);
  }
};
```

```
/*********************************************************************
***

Class: Scanner


The servo motor and ultrasonic sensor system

**********************************************************************
**/

class Scanner {
  private:
    byte pinT, pinE, pinS;
    Servo myServo;

  public:
    int frontDistance = 0;
    int duration;

    Scanner(){}   // Class Constructor

    // Assign pins for the Ultrasonic Sensor and Servo Motor
    void attach(byte _pinT, byte _pinE, byte _pinS){
```



```
      pinT = _pinT;
      pinE = _pinE;
      pinS = _pinS;

      pinMode(pinT, OUTPUT);
      pinMode(pinE, INPUT);
      myServo.attach(pinS);
    }

    // Calculate distance from obstacles
    // and returns the distance measured
    int getDistance(){
      digitalWrite(pinT, LOW);
```

```
    delayMicroseconds(2);
    digitalWrite(pinT, HIGH);
    delayMicroseconds(10);
    digitalWrite(pinT, LOW);

    int duration = pulseIn(pinE, HIGH);


int distance = duration * 0.034 / 2;   // cm
    return distance;
  }

  // rotate the scanner to a specified angle
  void scan(byte angle){
    myServo.write(angle);
    frontDistance = getDistance();
    delay(200);
  }
};
```

Source Code for the Controller Website:

```
/**********************************************************************
***

Include external libraries

**********************************************************************
**/

#include "esp_camera.h"
#include <WiFi.h>
#include "esp_timer.h"
#include "img_converters.h"
#include "Arduino.h"
#include "fb_gfx.h"
#include "soc/soc.h"           // disable brownout problems
#include "soc/rtc_cntl_reg.h"  // disable brownout problems
#include "esp_http_server.h"
```

```
/*********************************************************************
***

Constants definition for direction commands

*********************************************************************
**/


#define cmd_AutoOn          1
#define cmd_AutoOff         2
#define cmd_PumpOn          4
#define cmd_PumpOff         5
#define cmd_Forward         212
#define cmd_Backward        43
#define cmd_Left            135
#define cmd_Right           120
#define cmd_rotateLeft      153
#define cmd_rotateRight     102
#define cmd_Stop            0
#define cmd_UpperLeft       132
#define cmd_UpperRight      80
#define cmd_LowerLeft       3
#define cmd_LowerRight      40

// ESP32 WiFi connection credentials
const char* ssid     = "ESP32-Access-Point";
const char* password = "";

#define PART_BOUNDARY "123456789000000000000987654321"

/*********************************************************************
***

CAMERA_MODEL_AI_THINKER

Define pin assignments

*********************************************************************
**/
```

```
#define PWDN_GPIO_NUM     32
#define RESET_GPIO_NUM    -1
#define XCLK_GPIO_NUM      0
#define SIOD_GPIO_NUM     26
#define SIOC_GPIO_NUM     27

#define Y9_GPIO_NUM       35


#define Y8_GPIO_NUM       34
#define Y7_GPIO_NUM       39
#define Y6_GPIO_NUM       36
#define Y5_GPIO_NUM       21
#define Y4_GPIO_NUM       19
#define Y3_GPIO_NUM       18
#define Y2_GPIO_NUM        5
#define VSYNC_GPIO_NUM    25
#define HREF_GPIO_NUM     23
#define PCLK_GPIO_NUM     22

/*********************************************************************
***

HTTP stream content type and boundary definitions

**********************************************************************
**/

static const char* _STREAM_CONTENT_TYPE =
"multipart/x-mixed-replace;boundary=" PART_BOUNDARY;
static const char* _STREAM_BOUNDARY = "\r\n--" PART_BOUNDARY "\r\n";
static const char* _STREAM_PART = "Content-Type: image/jpeg\r\nContent-Length:
%u\r\n\r\n";

// HTTP server handles for camera and streaming
httpd_handle_t camera_httpd = NULL;
httpd_handle_t stream_httpd = NULL;
```

```
/*********************************************************************
***

HTML user interface for controlling the robot

*********************************************************************
**/

static const char PROGMEM INDEX_HTML[] = R"rawliteral(
<html>


	<head>
		<title>
			FumiBot Controller Page
		</title>
		<meta name = "viewport" content = "width = device-width, initial-scale =
1">
		<style>
    body {
				margin: 0;
				padding: 0;
				height: 100vh;
				font-family: Verdana, Geneva, sans-serif;
				text-align: center;
    }
    img {
				width: auto;
				max-width: 100%;
				height: auto;
    }
    table {
				float: left;
				margin: auto;
    }
			h1 {
				display: inline-block;
			}
    td {
				font-size: 15px;
```

```css
        font-weight: bold;
        padding: 8px;
}
button {
        margin: 10px;
        border: 1px solid black;
        background-color: white;
        color: black;
        align-items: center;
        justify-content: center;
        text-decoration: none;


        display: inline-flex;
        font-size: 18px;
        cursor: pointer;
        -webkit-touch-callout: none;
        -webkit-user-select: none;
        -khtml-user-select: none;
        -moz-user-select: none;
        -ms-user-select: none;
        user-select: none;
        -webkit-tap-highlight-color: rgba(0,0,0,0);
}
.button-circle {
        width: 50px;
        height: 50px;
        border-radius: 50%;
}
.button-left-semicircle {
        width: 30px;
        height: 50px;
        border-radius: 100% 0 0 100%/50% 0 0 50%;
}
.button-right-semicircle {
        width: 30px;
        height: 50px;
        border-radius: 0 100% 100% 0/0 50% 50% 0;
}
.switch {
```

```css
    position: relative;
    display: inline-block;
    width: 50px;
    height: 24px;
}
.switch input {
    opacity: 0;
    width: 0;
    height: 0;
}
.slider {
    position: absolute;


    cursor: pointer;
    top: 0;
    left: 0;
    right: 0;
    bottom: 0;
    background-color: #ccc;
    -webkit-transition: .4s;
    transition: .4s;
}
.slider:before {
    position: absolute;
    content: "";
    height: 16px;
    width: 16px;
    left: 4px;
    bottom: 4px;
    background-color: white;
    -webkit-transition: .4s;
    transition: .4s;
}
input:checked + .slider {
    background-color: #2196F3;
}
input:focus + .slider {
    box-shadow: 0 0 1px #2196F3;
}
```

```
    input:checked + .slider:before {
                                        -webkit-transform: translateX(26px);
                                        -ms-transform: translateX(26px);
                                        transform: translateX(26px);
    }
                        </style>
                </head>
                <body>
   <h1>
    FumiBot Controller
                        </h1>
                        <button class = "button-circle" onClick =


"window.open('https://egpc2009.github.io/About-FumiBot/', '_blank')">
                                i
                        </button>
   <br>
                        <table>
    <tr>
                                        <td align = "right">
        <label class = "switch">
          <input type = "checkbox" id = "auto" onClick = "setState('auto')">
          <span class = "slider"></span>
        </label>
                                        </td>
                                        <td>
                                                AUTO MODE
                                        </td>
    </tr>
    <tr>
                                        <td align = "right">
                                                <label class = "switch">
                                                        <input type = "checkbox" id = "pump"
onClick = "setState('pump')">
                                                        <span class = "slider"></span>
                                                </label>
                                        </td>
                                        <td>
```

```
Water Pump
</td>
</tr>
<tr>
<td align = "center">
<button class = "button-circle" onmousedown =
"toggleCheckbox('upperLeft');" ontouchstart = "toggleCheckbox('upperLeft');"
onmouseup = "toggleCheckbox('stop');" ontouchend = "toggleCheckbox('stop');">
&#8598;
</button>
</td>
<td align = "center">
<button class = "button-circle" onmousedown =
"toggleCheckbox('forward');" ontouchstart = "toggleCheckbox('forward');" onmouseup =

"toggleCheckbox('stop');" ontouchend = "toggleCheckbox('stop');">
&uarr;
</button>
</td>
<td align = "center">
<button class = "button-circle" onmousedown =
"toggleCheckbox('upperRight');" ontouchstart = "toggleCheckbox('upperRight');"
onmouseup = "toggleCheckbox('stop');" ontouchend = "toggleCheckbox('stop');">
&#8599;
</button>
</td>
</tr>
<tr>
<td align = "center">
<button class = "button-circle" onmousedown =
"toggleCheckbox('left');" ontouchstart = "toggleCheckbox('left');" onmouseup =
"toggleCheckbox('stop');" ontouchend = "toggleCheckbox('stop');">
&larr;
</button>
</td>
<td align = "center">
<button class = "button-left-semicircle"
onmousedown = "toggleCheckbox('rotateLeft');" ontouchstart =
```

```
"toggleCheckbox('rotateLeft');" onmouseup = "toggleCheckbox('stop');" ontouchend =
"toggleCheckbox('stop');">
                                        &#8634;
                                </button>
                                <button class = "button-right-semicircle"
onmousedown = "toggleCheckbox('rotateRight');" ontouchstart =
"toggleCheckbox('rotateRight');" onmouseup = "toggleCheckbox('stop');" ontouchend =
"toggleCheckbox('stop');">
                                        &#8635;
                                </button>
                        </td>
                        <td align = "center">
                                <button class = "button-circle" onmousedown =
"toggleCheckbox('right');" ontouchstart = "toggleCheckbox('right');" onmouseup =
"toggleCheckbox('stop');" ontouchend = "toggleCheckbox('stop');">


                                        &rarr;
                                </button>
                        </td>
                </tr>
  <tr>
                        <td align = "center">
                                <button class = "button-circle" onmousedown =
"toggleCheckbox('lowerLeft');" ontouchstart = "toggleCheckbox('lowerLeft');"
onmouseup = "toggleCheckbox('stop');" ontouchend = "toggleCheckbox('stop');">
                                        &#8601;
                                </button>
                        </td>
                        <td align = "center">
                                <button class = "button-circle" onmousedown =
"toggleCheckbox('backward');" ontouchstart = "toggleCheckbox('backward');"
onmouseup = "toggleCheckbox('stop');" ontouchend = "toggleCheckbox('stop');">
                                        &darr;
                                </button>
                        </td>
                        <td align = "center">
                                <button class = "button-circle" onmousedown =
"toggleCheckbox('lowerRight');" ontouchstart = "toggleCheckbox('lowerRight');"
onmouseup = "toggleCheckbox('stop');" ontouchend = "toggleCheckbox('stop');">
```

```
                                        &#8600;
                                    </button>
                                </td>
                            </tr>
                        </table>
                        <img src="" id="photo" >
                        <script>
                            function setState(element){
                                var checkbox = document.getElementById(element)
                                if(checkbox.checked == true){
                                    toggleCheckbox(element.concat("On"))
                                } else {
                                    toggleCheckbox(element.concat("Off"))
                                }




                            }
                                function toggleCheckbox(x) {
                                var xhr = new XMLHttpRequest();
                                xhr.open("GET", "/action?go=" + x, true);
                                xhr.send();
                            }

                            window.onload = function () {
                                document.getElementById("photo").src =
window.location.href.slice(0, -1) + ":81/stream";
                            }
                        </script>
                    </body>
</html>
)rawliteral";

// HTTP handler for serving the HTML page
static esp_err_t index_handler(httpd_req_t *req){
  httpd_resp_set_type(req, "text/html");
  return httpd_resp_send(req, (const char *)INDEX_HTML, strlen(INDEX_HTML));
}
```

```
// HTTP handler for streaming camera frames
static esp_err_t stream_handler(httpd_req_t *req){
  camera_fb_t * fb = NULL;
  esp_err_t res = ESP_OK;
  size_t _jpg_buf_len = 0;
  uint8_t * _jpg_buf = NULL;
  char * part_buf[64];

  res = httpd_resp_set_type(req, _STREAM_CONTENT_TYPE);
  if(res != ESP_OK){
    return res;
  }

  while(true){
    fb = esp_camera_fb_get();
    if (!fb) {



Serial.println("Camera capture failed");
      res = ESP_FAIL;
    } else {
      if(fb->width > 400){
        if(fb->format != PIXFORMAT_JPEG){
          bool jpeg_converted = frame2jpg(fb, 80, &_jpg_buf, &_jpg_buf_len);
          esp_camera_fb_return(fb);
          fb = NULL;
          if(!jpeg_converted){
            Serial.println("JPEG compression failed");
            res = ESP_FAIL;
          }
        } else {
          _jpg_buf_len = fb->len;
          _jpg_buf = fb->buf;
        }
      }
    }
    if(res == ESP_OK){
      size_t hlen = snprintf((char *)part_buf, 64, _STREAM_PART, _jpg_buf_len);
      res = httpd_resp_send_chunk(req, (const char *)part_buf, hlen);
```

```
    }
    if(res == ESP_OK){
      res = httpd_resp_send_chunk(req, (const char *)_jpg_buf, _jpg_buf_len);
    }
    if(res == ESP_OK){
      res = httpd_resp_send_chunk(req, _STREAM_BOUNDARY,
strlen(_STREAM_BOUNDARY));
    }
    if(fb){
      esp_camera_fb_return(fb);
      fb = NULL;
      _jpg_buf = NULL;
    } else if(_jpg_buf){
      free(_jpg_buf);
      _jpg_buf = NULL;
    }
    if(res != ESP_OK){



break;
    }
    //Serial.printf("MJPG: %uB\n",(uint32_t)(_jpg_buf_len));
  }
  return res;
}

// HTTP handler for processing commands from the web interface
static esp_err_t cmd_handler(httpd_req_t *req){
  char*  buf;
  size_t buf_len;
  char variable[32] = {0,};

  buf_len = httpd_req_get_url_query_len(req) + 1;
  if (buf_len > 1) {
    buf = (char*)malloc(buf_len);
    if(!buf){
      httpd_resp_send_500(req);
      return ESP_FAIL;
    }
```

```
    if (httpd_req_get_url_query_str(req, buf, buf_len) == ESP_OK) {
      if (httpd_query_key_value(buf, "go", variable, sizeof(variable)) == ESP_OK) {
      } else {
        free(buf);
        httpd_resp_send_404(req);
        return ESP_FAIL;
      }
    } else {
      free(buf);
      httpd_resp_send_404(req);
      return ESP_FAIL;
    }
    free(buf);
  } else {
    httpd_resp_send_404(req);
    return ESP_FAIL;
  }



sensor_t * s = esp_camera_sensor_get();
  int res = 0;

  s->set_brightness(s, 2);
  s->set_vflip(s, 1);

  // Handle commands received from the web interface
  if(!strcmp(variable, "autoOn")) {
    Serial.write(cmd_AutoOn);
  }
  else if(!strcmp(variable, "autoOff")) {
    Serial.write(cmd_AutoOff);
  }
  else if(!strcmp(variable, "pumpOn")) {
    Serial.write(cmd_PumpOn);
  }
  else if(!strcmp(variable, "pumpOff")) {
    Serial.write(cmd_PumpOff);
  }
```

```
  else if(!strcmp(variable, "forward")) {
    Serial.write(cmd_Forward);
  }
  else if(!strcmp(variable, "backward")) {
    Serial.write(cmd_Backward);
  }
  else if(!strcmp(variable, "left")) {
    Serial.write(cmd_Left);
  }
  else if(!strcmp(variable, "right")) {
    Serial.write(cmd_Right);
  }
  else if(!strcmp(variable, "rotateLeft")) {
    Serial.write(cmd_rotateLeft);
  }
  else if(!strcmp(variable, "rotateRight")) {
    Serial.write(cmd_rotateRight);
  }
  else if(!strcmp(variable, "upperLeft")) {
    Serial.write(cmd_UpperLeft);


}
  else if(!strcmp(variable, "upperRight")) {
    Serial.write(cmd_UpperRight);
  }
  else if(!strcmp(variable, "lowerLeft")) {
    Serial.write(cmd_LowerLeft);
  }
  else if(!strcmp(variable, "lowerRight")) {
    Serial.write(cmd_LowerRight);
  }
  else if(!strcmp(variable, "stop")) {
    Serial.write(cmd_Stop);
  }
  else {
    res = -1;
  }

  if(res){
```

```
    return httpd_resp_send_500(req);
  }

  httpd_resp_set_hdr(req, "Access-Control-Allow-Origin", "*");
  return httpd_resp_send(req, NULL, 0);
}

// Start the camera HTTP server
void startCameraServer(){
  httpd_config_t config = HTTPD_DEFAULT_CONFIG();
  config.server_port = 80;
  httpd_uri_t index_uri = {
    .uri       = "/",
    .method    = HTTP_GET,
    .handler   = index_handler,
    .user_ctx  = NULL
  };

  httpd_uri_t cmd_uri = {
    .uri       = "/action",
    .method    = HTTP_GET,

.handler   = cmd_handler,
    .user_ctx  = NULL
  };
  httpd_uri_t stream_uri = {
    .uri       = "/stream",
    .method    = HTTP_GET,
    .handler   = stream_handler,
    .user_ctx  = NULL
  };
  if (httpd_start(&camera_httpd, &config) == ESP_OK) {
    httpd_register_uri_handler(camera_httpd, &index_uri);
    httpd_register_uri_handler(camera_httpd, &cmd_uri);
  }
  config.server_port += 1;
  config.ctrl_port += 1;
  if (httpd_start(&stream_httpd, &config) == ESP_OK) {
    httpd_register_uri_handler(stream_httpd, &stream_uri);
  }
```

```
}

void setup() {
  WRITE_PERI_REG(RTC_CNTL_BROWN_OUT_REG, 0); //disable brownout
detector

  Serial.begin(115200);
  Serial.setDebugOutput(false);

  camera_config_t config;
  config.ledc_channel = LEDC_CHANNEL_0;
  config.ledc_timer = LEDC_TIMER_0;
  config.pin_d0 = Y2_GPIO_NUM;
  config.pin_d1 = Y3_GPIO_NUM;
  config.pin_d2 = Y4_GPIO_NUM;
  config.pin_d3 = Y5_GPIO_NUM;
  config.pin_d4 = Y6_GPIO_NUM;
  config.pin_d5 = Y7_GPIO_NUM;
  config.pin_d6 = Y8_GPIO_NUM;
  config.pin_d7 = Y9_GPIO_NUM;
  config.pin_xclk = XCLK_GPIO_NUM;


config.pin_pclk = PCLK_GPIO_NUM;
  config.pin_vsync = VSYNC_GPIO_NUM;
  config.pin_href = HREF_GPIO_NUM;
  config.pin_sscb_sda = SIOD_GPIO_NUM;
  config.pin_sscb_scl = SIOC_GPIO_NUM;
  config.pin_pwdn = PWDN_GPIO_NUM;
  config.pin_reset = RESET_GPIO_NUM;
  config.xclk_freq_hz = 20000000;
  config.pixel_format = PIXFORMAT_JPEG;

  if(psramFound()){
    config.frame_size = FRAMESIZE_VGA;
    config.jpeg_quality = 10;
    config.fb_count = 2;
  } else {
    config.frame_size = FRAMESIZE_SVGA;
    config.jpeg_quality = 12;
```

```
    config.fb_count = 1;
  }

  // Initialize Camera
  esp_err_t err = esp_camera_init(&config);
  if (err != ESP_OK) {
    Serial.printf("Camera init failed with error 0x%x", err);
    return;
  }
  // Setup AP
  Serial.print("Setting AP (Access Point)…");
  // Remove the password parameter, if you want the AP (Access Point) to be open
  WiFi.softAP(ssid, password);

  IPAddress IP = WiFi.softAPIP();
  Serial.print("Camera Stream Ready! Connect to the ESP32 AP and go to: http://");
  Serial.println(IP);

  // Start streaming web server
  startCameraServer();

}


void loop() {
  }
```

5.4.2: Appendix B. Budget:

Components	Quantity	Total
Arduino Uno R3 Compatible Board	1	₱256
L293D Motor Control Shield	1	₱113

ESP32S with Cam & External Antenna	1	₱298.79
12V Battery	1	₱100
Ultrasonic Sensor	1	₱39
Servo Motor (180°)	1	₱79
Hobby Gearmotor	4	₱39
Disinfectant Container/Storage	1	₱25
Water Pump	1	₱89
Nozzle	1	₱85
Rectangular Mop Head	1	₱55
Jumper Wire	80	₱79
Resistor	1	₱70
Filament	1	₱300
Cable Tie	1	₱15
Hose	1	₱8

5.4.3 Appendix C. Copy of Invitation Letter:

February 7,2025

Saint Joseph School Naga City

(Name Of Panelist)

Subject: Invitation to Serve as a Panelist for Our ICT Research Defense

Dear (Mme/Sir Name of Panelist),

I hope this letter finds you well. On behalf of our research team, We are pleased to invite you to serve as a panelist for our upcoming defense presentation for our ICT research project titled FumiBot V2: An Innovative Cleaning Solution for Flat Surface Mopping.

The defense is scheduled to take place on February 18,2025 at 2:40pm-3:30pm

As a panelist, we kindly ask for your insights and evaluation of our study based on its objectives, methodology, findings, and overall contribution to the field of ICT

Your presence would greatly contribute to the success of our research defense.
We would be honored to have you with us and appreciate your time and consideration.

Thank you for your time and support.

Sincerely,

Project Manager
Janaya Therese G. Penilla

Project Manager
Rover Rhys T. Uy

Noted By:

REYMARK A. REGLOS, LPT

VENUS DIANNE MARIE B. MALENIZA, LPT, MAEd
Robotics Instructor

Science Area Coordinator

Rationale

In this increasingly fast-paced world, where time is money, there has been an increase in demand for automated cleaning solutions. Due to the amount of cleaning tasks, faced in homes, workplaces, and others, maintaining cleanliness in residential, commercial, and industrial environments poses a significant challenge. The IFR (International Federation of Robotics) has seen a surge in the use of Service Robots, which can destroy 99% of microorganisms on surfaces within 10 minutes. Cleaning bots are being utilized in hotels and public spaces for safety during the spreading of diseases by minimizing human contact.

It includes a comparative study on the effectiveness of autonomous cleaning robots in reducing hospital-acquired infections (HAIs) in healthcare settings. This study analyzes data from various facilities and finds that robots significantly reduce HAI incidence, potentially saving lives and reducing healthcare costs. With the help of ICE Robotics, a leading technology and cleaning equipment company, robotic technology is helping healthcare facilities by helping in infection control, and patient safety. Trends include autonomous floor cleaning robots, UV disinfection robots, and data-driven analytics.

Dirt, sand, and dust can be cleaned easily with the traditional method of sweeping the floor with a broom or dust mop or using a vacuum cleaner for more efficient and easier cleaning. Nevertheless, cleaning away what can be seen on surfaces would be inadequate. There would frequently be bacteria, viruses, and such harboring in these surfaces, which can not be eliminated simply by sweeping or vacuuming. In these current times, new infectious diseases emerged such as monkeypox, avian influenza, and other zoonotic diseases. Moreover, bacteria and viruses have developed new mutations and increased resistance. To be safe from these diseases, these microorganisms must be eradicated through special methods such as disinfecting.

Following this, due to having limited time while having a load of cleaning tasks, robotic cleaning technologies have progressed at an increasing rate, from simple devices to highly complicated robots capable of handling the most challenging tasks. Cleaning robots started as having constrained capabilities, mostly designed for basic tasks such as vacuuming surfaces like carpets. However, as time passes, there has been an escalating need for more advanced cleaning robots, which drove the research and development of robots with numerous skill sets.

According to the ICE Cobotics,this can result in owners of these advanced cleaning robots can have more time to run errands and other important tasks. They no longer need to consume extended amounts of time cleaning. The Fumibot V2 can also decrease the need for human cleaning labor or hiring cleaners. Subsequently, operational costs would be lower and there would be an increase in productivity. This could play a crucial role in preventing the spread of infections and diseases. The advantages of disinfectant robots offer reproducibility and quality assurance by recording operation parameters and they include unmanned, standardized

disinfection without human presence, reducing exposure to harmful UV radiation, providing additional hygiene benefits, and leaving no residues, making it an environmentally friendly disinfectant method.

Treatment / Procedure

Conceptualizing the 3D Design of the Robot

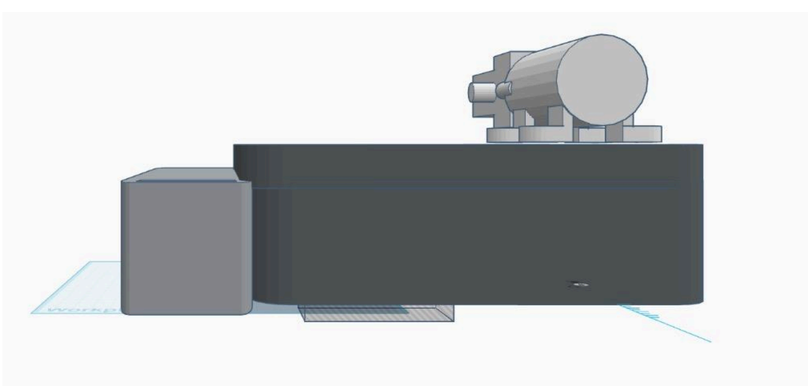

Figure 2.1 Side View of the Robot

Figure 2.2 Top Side View of the Robot

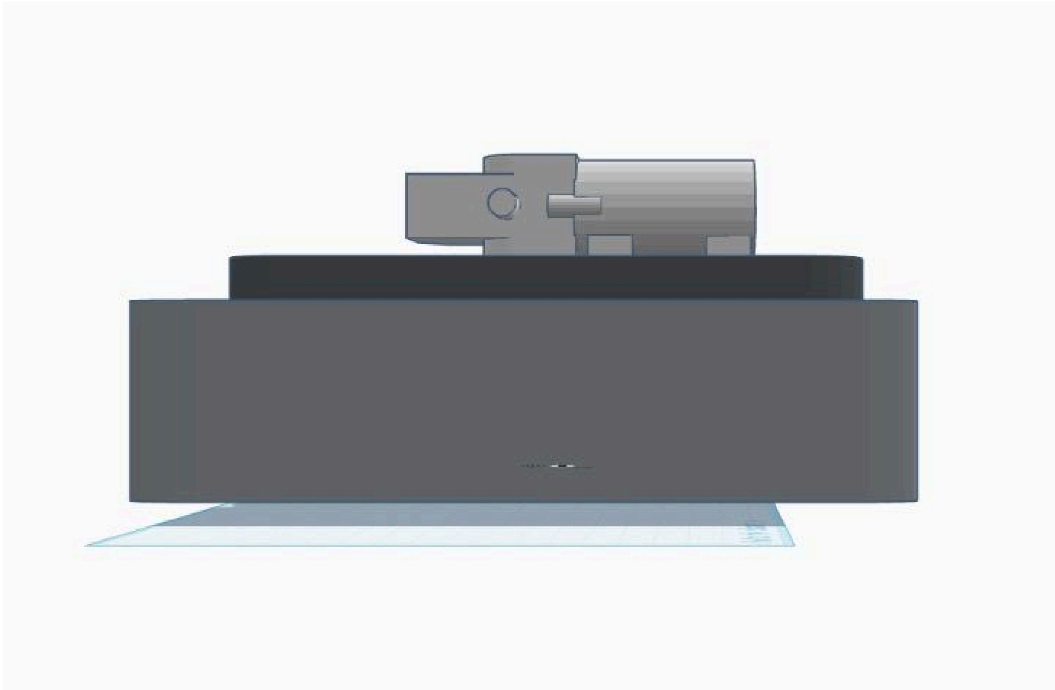

Figure 2.3 Back Side View of the Robot

As seen in the figures above, the cleaning solution for flat surfaces is composed of multiple parts. This includes the main control system, movement portion, and disinfecting portion.

The main control system includes the Arduino Uno R3 Compatible Board, Motor Control Shield, ESP32S with Cam & External Antenna, and the 12V battery. These components are located within the bot since the first three are fragile and sensitive, to avoid complications. The Arduino Uno R3 Compatible Board and the Motor Control Shield will be the main control of all the components to be used, and the motor control for motor movements. The ESP32S with CamMB & External Antenna is the main communication module and camera, so there is a hole for the camera to view from and a gap for the antenna to pass through. Lastly, the 12V Battery, used to power all the electrical components in the robot, is located inside to prevent short-circuiting from occurring due to coming in contact with the sprayed disinfectant liquid.

The movement portion comprises the Ultrasonic Sensor, Servo Motor (180°), and Hobby Gearmotors. The Ultrasonic Sensor and Servo Motor (180°) are located in front of the robot while the Hobby Gearmotors are at the bottom part of the robot. The Servo Motor (180°) turns the Ultrasonic Sensor from left to right to prevent the robot from hitting any obstacles within the 180° range in front of the robot. The Hobby Gear Motors are used to control the wheels.

The disinfecting portion includes the disinfectant container, water pump, hose, nozzle, and rectangular mop head. The disinfectant container is on the top of robot, and a hose connected to a water pump. A hose is connected to the container to bring the liquid to the water pump. The liquid will then get pumped out through another hose to the nozzle to be sprayed on the surface to disinfect. This will then get wiped by the rectangular mop head.

Listing the Components of the Robot

With the 3D design formulated, the Project Designers will first research the materials and components that may be used for constructing the cleaning solution for flat surfaces. They will then validate that these materials and components are appropriate for the robot. After that, they will list all the materials and components needed.

Gathering the Components for the Robot

The Project Managers and the assigned group members will then purchase the

components necessary to assemble the cleaning solution. They will buy all the required components ahead of time, after approval. Moreover, they will check the components after delivery to ensure no damages or missing materials. In this manner, a straightforward assembling process will be ensured.

Constructing the Robot

The group, mainly the Project Designers, will assemble the cleaning solution for flat surfaces. The group ensures that no wires are tangled, components will not get wet or damaged during operations, and that the bot will be assembled properly. Most importantly, the Project Managers will guide and check the assembly procedures done by the group.

Coding

To operate the components used in the cleaning solution bot, the Project Designers will utilize the Arduino C++ programming language. Arduino C++ is a specialized version of C++

programming language with a unique set of added functions and libraries, and many more things that extend its usefulness ahead of those of standard C++. It provides freedom to create a wide range of electronic systems, such as the bot.

Multiple portions of the FumiBot V2.0 require programming. One crucial portion is to prevent the robot from hitting obstacles, which involves the Ultrasonic Sensor, the Arduino Uno R3 Compatible Board, and the ESP32S with CamMB & Antenna. Another portion is for finding out the amount of disinfectant liquid left in the container, which includes the Water Level Detection Sensor Module. The last crucial portion is for controlling the robot wirelessly and seeing the robot's perspective, which includes the ESP32S with CamMB & Antenna.

Inspecting the Finished Robot

After the construction and coding of the FumiBot V2, the Project Managers will double-check the overall construction and durability of the FumiBot V2 to ensure that it will work properly during operations. Ensuring that there are no skipped procedures in assembling and coding the robot. Moreover, the constructive criticisms, given by trusted outside sources, will also be taken into account. Furthermore, the robot will be tested from time to time for problems.

Timetable

TASKS	TARGET DATE	TEACHER'S DEADLINE
1. Submission of Problem and Title	12/14/2024	9/2/2024
2. Writing the IP/Arduino Project Proposal	9/2/2024	9/2/2024
3. Submission of IP/Arduino Project Proposal	9/2/2024	9/2/2024
4. Investigation Period	9/2/2024	9/2/2024
5. Submission of Gathered Data	9/2/2024	9/2/2024

6. Writing the IP/Arduino Project Written Report	12/12/2024	12/12/2024
7. Submission of IP/Arduino Project Written Report	12/14/2024	12/14/2024

References

1. https://doi.org/10.1186/s13756-020-00878-4*cs*. (n.d.). IFR International Federation of *Robotics*. https://ifr.org/case-studies/cleaning-robots-reduce-infections

2. Al-Farsi, A., & Hassan, T. (2023, November 20). *Evaluation of Autonomous Cleaning Robots in Reducing Hospital Acquired Infections: A Comparative Study*. https://vectoral.org/index.php/IJSICS/article/view/77/

3. Robotic *Technology: Cleaning Trends for Healthcare in 2023*. (n.d.). ICE Cobotics.

   https://icecobotics.com/blog/robotic-technology-clhttps://vectoral.org/index.php/IJSICS/article/view/77/

4. Schahawi, M. D. E., Zingg, W., Vos, M., Humphreys, H., Lopez-Cerero, L., Fueszl, A., Zahar, J. R., & Presterl, E. (2021). Ultraviolet disinfection robots to improve hospital cleaning: Real promise or just a gimmick? *Antimicrobial Resistance and Infection Control*, *10*(1). https://doi.org/10.1186/s13756-020-00878-4

Name: Project Manager - Rover Uy

Name	Rating	Reason
Rover Uy	5/5	Greatly contributed to the making of the proposal. Always initiates on having meetings for the whole group.
Janaya Penilla	5/5	Janaya greatly contributed to the making of the proposal.
Noah Pereña	3/5	Noah contributed some ideas but didn't really help in the docs and on issues faced.
Ryza Carcido	5/5	Ryza Contributes ideas and does the tasks given to her
Misha Medroso	5/5	Misha Contributes ideas and does the tasks given
Roma Peyra	5/5	RomaContributes ideas and does the tasks given to her
Josiah Tomosada	5/5	Josiah Contributes ideas and does the tasks given to him
Rain Dimayuga	5/5	RainContributes ideas and does the tasks given to him
Ethan Co	5/5	Ethan Co Went way above the needed of him and gave so much more very much deserves his score our humblest king

Name: Project Manager - Janaya Penilla

Name	Rating	Reason
Rover Uy	5/5	Contributed most to the project proposal and suggested more ideas to improve the bot.
Janaya Penilla	5/5	Guided the document managers in doing their assigned tasks and reminded the team of the important details of the project proposal.
Noah Pereña	5/5	Shows effort in contributing to the preparation of the project proposal document and bot.
Ryza Carcido	5/5	Presents ideas for the proposed bot in person and participates in group discussions.
Misha Medroso	5/5	Accomplishes the given tasks assigned to her and double-checks for any grammatical errors in the document.
Roma Peyra	5/5	Initiates the group meetings and attentively follows the assigned takes given as well as checks the document for any missing information.
Josiah Tomosada	5/5	Actively participates in group discussions and engages ideas for all to agree on the proposed bot.
Rain Dimayuga	5/5	Contributes and suggests ideas for the bot and does the assigned tasks for the proposal.
Ethan Co	5/5	Guided the document managers and project designers to construct the proposal document and contributed most of the work assigned.

Name: Document Manager & Researcher - Noah Pereña

Name	Rating	Reason
Rover Uy	5/5	He do his tasks greatly and contributed to the proposal
Janaya Penilla	5/5	She helped and do her tasks greatly
Noah Pereña	5/5	He do all of his tasks and show some ideas
Ryza Carcido	5/5	She do her tasks and provide ideas about the proposal
Misha Medroso	5/5	She contributed and do her tasks in the proposal
Roma Peyra	5/5	She participated in discussions and contributed her tasks
Josiah Tomosada	5/5	He helped making the bot designs and contributed in the proposal
Rain Dimayuga	5/5	He helped designed the bot and contributed in the proposal
Ethan Co	5/5	He contributed his tasks and help in the proposal

Name: Document Manager & Researcher - Ryza Carcido

Name	Rating	Reason
Rover Uy	5/5	He actively participated and called for meetings, and he greatly communicates with our groupmates.
Janaya Penilla	5/5	Janaya delivered high-quality writing and editing, ensuring the proposal was clear amd concise
Noah Pereña	4/5	Noah partially participated in the proposal. He provided ideas in the robot itself.
Ryza Carcido	4/5	Partially contributed valuable ideas during brainstorming sessions. Managed to do her part in the proposal
Misha Medroso	4/5	Misha completed her assigned task, made some changes and the proposal. Partially contributed to brainstorming ideas during meetings.
Roma Peyra	5/5	Roma actively participated in discussions and meetings as well as maintaining a good collaboration with the team members
Josiah Tomosada	5/5	Josiah exhibited great collaboration and communication with the members of the group and integrated everyone’s contributions
Rain Dimayuga	5/5	Rain provided thorough research and analysis, offering valuable insights for the proposal’s content
Ethan Co	5/5	Ethan showed strong leadership and organizational skills, ensuring that all tasks were

		completed on time.

Name: Document Manager & Researcher - Misha Medroso

Name	Rating	Reason
Rover Uy	5/5	Conducted all the meetings and was persistent on making sure everything was being done on time.
Janaya Penilla	5/5	Actively participates in group discussion and does her job as project manager.
Noah Pereña	4/5	Did the tasks that were given.
Ryza Carcido	4/5	Did the tasks that were given.
Misha Medroso	4/5	Did the tasks that were given.
Roma Peyra	5/5	Greatly contributed to the proposal and actively shares her ideas.
Josiah Tomosada	5/5	Greatly contributed to the proposal and actively shares his ideas.
Rain Dimayuga	5/5	Greatly contributed to the proposal and actively shares his ideas.
Ethan Co	5/5	Greatly contributed to the proposal and actively shares his ideas.

Name: Document Manager & Researcher - Roma Peyra

Name	Rating	Reason
Rover Uy	5/5	Greatly contributed to the making of the proposal. Always initiates having meetings for the whole group.
Janaya Penilla	5/5	Greatly contributed to the making of the proposal, and shared ideas that helped create the proposal.
Noah Pereña	4/5	Did all the tasks assigned to him however he is sometimes absent in meetings.
Ryza Carcido	5/5	She did the tasks assigned to her.
Misha Medroso	5/5	She completed all of the tasks assigned to her.
Roma Peyra	5/5	She completed all of the tasks assigned to her.
Josiah Tomosada	5/5	He is cooperative and is hands-on in making the design for the bot.
Rain Dimayuga	5/5	He is hands-on in making the design of the bot and always considers other members' thoughts about the design.
Ethan Co	5/5	Showed lots of effort in making the proposal. He also greatly contributed to making the proposal

Name: Project Designer - Josiah Tomosada

Name	Rating	Reason
Rover Uy	5/5	He is always active in the making of our proposal, ensuring that everyone does their part.
Janaya Penilla	5/5	She shared her ideas that contributed to our proposal.
Noah Pereña	5/5	He shared ideas and did his part in the proposal with effort.
Ryza Carcido	5/5	She completed her part in the proposal with effort and actively participated during meetings
Misha Medroso	5/5	She completed her part of the proposal with effort and always shared ideas.
Roma Peyra	5/5	She participates actively in meetings and always reminds us of what we will do.
Josiah Tomosada	5/5	Helped in making the design of the bot and did my part in the proposal
Rain Dimayuga	5/5	Helped in making the design of the bot and did his part in the proposal.
Ethan Co	5/5	Always and actively gives effort in the proposal. He also helps us with things we are having a hard time with.

Name: Project Designer - Rain Dimayuga

Name	Rating	Reason
Rover Uy	5/5	Has shown his best even when we had other tasks and whatnot still put in the time to join in a call and discuss.
Janaya Penilla	5/5	Active in group meetings and contributed greatly in the proposal.
Noah Pereña	5/5	Contributed in finishing the proposal on time.
Ryza Carcido	5/5	Accomplishes her tasks efficiently and in a timely manner.
Misha Medroso	5/5	Finishes the tasks given swiftly.
Roma Peyra	5/5	Contributed in finishing the proposal and recorded the minutes of each meeting
Josiah Tomosada	5/5	Active in group meetings and helped me in designing the bot.
Rain Dimayuga	5/5	Designed the 3D model of the bot.
Ethan Co	5/5	Contributed greatly in the making of the proposal, provided ideas for the design, active in group meetings.

Name: Project Designer - Ethan Co

Name	Rating	Reason
Rover Uy	5/5	Rover coordinated group meetings, either online or in person, and the equal distribution of tasks. He rechecked and made corrections to the proposal. He provided ideas for the robot.
Janaya Penilla	5/5	Janaya helped coordinate group meetings, either online or in person, and the equal distribution of tasks. She rechecked and made corrections to the proposal.
Noah Pereña	4/5	Noah partially contributed to the creation of the proposal. He provided some ideas for the robot. He participates in group meetings, in person.
Ryza Carcido	4/5	Ryza partially contributed to the creation of the proposal.
Misha Medroso	4/5	Misha partially contributed to the creation of the proposal.
Roma Peyra	5/5	Roma contributed to the creation of the proposal. She took down minutes for each group meeting. She participates in group meetings, both online and in person.
Josiah Tomosada	5/5	Josiah contributed to the creation of the proposal. He provided ideas for the robot. He formulated the 3D design of the FumiBot V2. He participates in group meetings, in person.
Rain Dimayuga	5/5	Rain contributed to the creation of the proposal. He provided ideas for the robot. He formulated the 3D design of the FumiBot V2. He participates in group

		meetings, in person.
Ethan Co	5/5	I contributed to the creation of the proposal. I provided ideas for the robot. I made the digital 3D versions of specific vital components for the FumiBot V2. I participate in group meetings, both online and in person.